HDOC Day-68

Decimal to Binary Conversion Using Recursion in C

Aim

To write a C program using recursion, a user-defined function, conditional statements, and arithmetic operations to convert a non-negative decimal integer into its binary representation without using iterative loops for the conversion.

Algorithm

1. Start the program and declare an integer variable n.
2. Read the decimal integer from the user.
3. If the input is negative, display an error message and terminate.
4. Define the recursive function decimalToBinary(int n).
5. If n is zero inside the recursive function, return (base case).
6. Otherwise, recursively call decimalToBinary(n / 2).
7. After the recursive call returns, print n % 2.
8. In main(), print 0 directly when the original input is zero; otherwise call decimalToBinary(n).
9. Print a newline and stop.

C Program

```
#include <stdio.h>

void decimalToBinary(int n);

int main()
{
    int n;

    printf("Enter a decimal number: ");
    scanf("%d", &n);

    if (n < 0)
    {
        printf("Invalid input! Enter a non-negative integer.\n");
        return 0;
    }

    printf("Binary Representation: ");

    if (n == 0)
        printf("0");
    else
        decimalToBinary(n);

    printf("\n");
    return 0;
}

void decimalToBinary(int n)
{
    if (n == 0)
        return;

    decimalToBinary(n / 2);
    printf("%d", n % 2);
}
```

Explanation

Decimal and binary: Decimal uses digits 0-9 (base 10), whereas binary uses 0 and 1 (base 2). Repeated division by 2 produces remainders that form the binary representation when read in reverse order.

Example: n = 13. The successive quotients and remainders are shown below.

Step	Number	Quotient	Remainder
1	13	6	1
2	6	3	0
3	3	1	1
4	1	0	1

Reading the remainders from bottom to top gives **1101**. Therefore, (13) base 10 = (1101) base 2.

How Recursion Works

The function calls itself with n / 2 until n becomes zero. It prints each remainder only after the deeper recursive call returns. This is why the binary digits appear in the correct order.

```
decimalToBinary(13)
  decimalToBinary(6)
    decimalToBinary(3)
      decimalToBinary(1)
        decimalToBinary(0)  // base case

Return and print: 1, then 1, then 0, then 1
Binary representation: 1101
```

Base case: if (n == 0) return. **Recursive case:** decimalToBinary(n / 2), followed by printing n % 2.

Complexity: O(log n) time and O(log n) auxiliary stack space for positive n. The value zero is handled directly in main().

Test Case Table

Test	Decimal input	Expected binary output
1	13	1101
2	10	1010
3	5	101
4	8	1000
5	15	1111
6	1	1
7	0	0
8	25	11001
9	100	1100100
10	-5	Invalid input

Sample Outputs

Test Case 1: Decimal 13

```
Enter a decimal number: 13
Binary Representation: 1101
```

Test Case 2: Decimal 10

```
Enter a decimal number: 10
Binary Representation: 1010
```

Test Case 3: Power of Two

```
Enter a decimal number: 8
Binary Representation: 1000
```

Test Case 4: Zero

```
Enter a decimal number: 0
Binary Representation: 0
```

Test Case 5: Invalid Input

```
Enter a decimal number: -5
Invalid input! Enter a non-negative integer.
```

Result

The C program uses a recursive user-defined function to convert a decimal integer into its binary representation. It repeatedly divides the number by two and prints the remainders while recursive calls return. The program handles positive integers, zero, and invalid negative inputs, demonstrating decimal-to-binary conversion without iterative loops.